LifeTech
ITS Academy

CORSO DI TECNICO SUPERIORE PER LO SVILUPPO E LA GESTIONE DI SOLUZIONI DI INFORMATICA BIOMEDICA (TIB) 2024-2026

Codice progetto: 2024/5173/0

UF 9.1 PROGRAMMAZIONE AD OGGETTI IN JAVA

DOCENTE: ALESSANDRO BISONTIN, SIMONE KRESEVIC

DATA: 23/03/2026

FUTURA
Finanziato dall'Unione europea NextGenerationEU
Ministero dell'Istruzione e del Merito
LA SCUOLA PER L'ITALIA DI DOMANI
PNRR ISTRUZIONE

Esercitazione: Clinica Triage 2.0

Durata: 5 ore (2 sessioni) — OOP + Collections + I/O + Eccezioni

UF9.1 – Programmazione ad Oggetti in Java

Consegna

Contesto

Realizzare un sistema di gestione triage per la clinica di un ospedale.

Il sistema deve gestire un archivio di più pazienti, ognuno con una storia di visite mediche.

L'archivio dei pazienti è persistente: viene caricato da file all'avvio e salvato alla chiusura.

Il sistema funziona tramite un menu da console (Scanner o BufferReader) e gestisce più pazienti contemporaneamente.

Argomenti richiesti

Tutto ciò fatto prima di oggi + ArrayList, HashMap, eccezioni custom (throws/try-catch), BufferedReader/PrintWriter (I/O su file), classi Wrapper (Integer.parseInt, Double), Math, LocalDate.

Tempo a disposizione: 6 ore (Sessione 1: 2h + Sessione 2: 4h + eventuale lavoro a casa)

Lavoro in team

DEADLINE: Lunedì 30\03\2026 09:00:00

Teams

Team 1

...

Team 2

...

Team 3

...

Team 4

...

Team 5

...

Team 6

...

Contesto

Il triage è una procedura infermieristica rapida e sistematica utilizzata nel Pronto Soccorso per valutare i pazienti all'arrivo, assegnando una priorità di trattamento (codice) basata sulla gravità clinica e sul rischio evolutivo, piuttosto che sull'ordine di arrivo.

L'obiettivo è identificare i casi critici e ottimizzare le cure dove un infermiere esperto valuta i parametri vitali e i sintomi per classificare il paziente assegnando dei codici di priorità (5 livelli).

Rosso (Emergenza)	Pericolo di vita immediato.
Arancione (Urgenza)	Paziente con patologie gravi a rischio evolutivo.
Azzurro (Urgenza differibile)	Condizioni stabili con rischio evolutivo.
Verde (Urgenza minore)	Patologie di modesta entità.
Bianco (Non urgenza)	Problemi non urgenti.

Cosa cambia dalla versione 1

PRIMA (v1)	ORA (v2)
• Un paziente alla volta	• **Archivio multi-paziente (HashMap)**
• Una sola visita per paziente	• **Storico visite per paziente (ArrayList)**
• Nessun salvataggio su file	• **Salvataggio/caricamento da file (I/O)**
• Validazione con if/else semplici	• **Eccezioni custom (try-catch + throws)**
• Data visita come String	• **Data con LocalDate + formattazione**
• Classe Main con tutto dentro	• **Classi Clinica + GestoreArchivio**

La struttura OOP (Visita astratta, 3 sottoclassi, Paziente) rimane la stessa — la evolviamo!

Requisiti funzionali

1. Gestire un archivio di più pazienti usando HashMap<String, Paziente> indicizzato per codice fiscale
2. Ogni paziente può avere più visite: ArrayList<Visita> al posto del singolo attributo Visita
3. Validare il CF con eccezione custom CodiceFiscaleNonValidoException (extends Exception). Gestire con try-catch
4. Cercare paziente per CF. Se non esiste, lanciare PazienteNonTrovatoException. Gestire con try-catch
5. Salvare l'archivio su file di testo (PrintWriter) e caricarlo all'avvio (BufferedReader). Formato custom definito nelle slide
6. Usare LocalDate per le date (registrazione e visite). Wrapper Integer.parseInt() per parsing file. *(???) Math per statistiche*
7. Stampare scheda completa con TUTTE le visite. *Statistiche: conteggio per colore, numero visite, media visite per paziente*
8. Menu interattivo con 9 opzioni: registra, cerca, aggiungi visita, scheda, elenca per colore, statistiche, salva, carica, esci

Diagramma delle Classi UML (1/2)

Paziente

- codiceFiscale: String
- nome: String
- cognome: String
- coloreTriage: String
- visite: ArrayList<Visita>
- dataRegistrazione: LocalDate

+ Paziente(cf, nome, cognome, colore)
+ getCodiceFiscale(): String
+ getNomeCompleto(): String
+ getColoreTriage(): String
+ getVisite(): ArrayList<Visita>
+ getDataRegistrazione(): LocalDate
+ aggiungiVisita(v: Visita): void
+ calcolaPriorita(): String
+ stampaScheda(): void

"se ha, ha una o molte"
0,N

<<abstract>>
Visita

- data: LocalDate
- medico: String
- diagnosi: String

+ Visita(data, medico, diagnosi)
+ getData(): LocalDate
+ getMedico(): String
+ getDiagnosi(): String
+ descriviVisita(): String {abstract}

extends (ereditarietà)

VisitaGenerica

- reparto: String

+ VisitaGenerica(...)
+ getReparto(): String
+ descriviVisita(): String

VisitaOrtopedica

- parteCorpo: String

+ VisitaOrtopedica(...)
+ getParteCorpo(): String
+ descriviVisita(): String

VisitaCardiologica

- frequenzaCardiaca: int
- pressioneSist: int
- pressioneDiast: int

+ VisitaCardiologica(...)
+ getFrequenza(): int
+ descriviVisita(): String

Diagramma delle Classi UML (2/2)

Clinica

```
- archivio: HashMap<String, Paziente>

+ Clinica()
+ registraPaziente(p: Paziente): void
+ cercaPaziente(cf: String): Paziente
    throws PazienteNonTrovatoException
+ getPazientiPerColore(colore: String): ArrayList<Paziente>
+ getArchivio(): HashMap<...>
+ stampaStatistiche(): void
```

GestoreArchivio

```
+ salvaArchivio(archivio: HashMap<...>, percorso: String): void
+ caricaArchivio(percorso: String): HashMap<String, Paziente>
```

PazienteNonTrovatoException

```
extends Exception
+ PazienteNonTrovatoException(msg: String)
```

CodiceFiscaleNonValidoException

```
extends Exception
+ CodiceFiscaleNonValidoException(msg: String)
```

Main

```
+ main(args: String[]): void
+ mostraMenu(): void
```

Dettagli implementativi

Classe Paziente

- Tutti gli attributi private con getter e setter
- visite è un ArrayList<Visita> (inizializzato vuoto)
- aggiungiVisita(v) fa visite.add(v)
- stampaScheda() itera su TUTTE le visite con for-each
- dataRegistrazione = LocalDate.now() nel costruttore

Metodo calcolaPriorita()

- Se visite vuote → "NON VISITATO"
- Guarda l'ULTIMA visita nella lista
- Rosso + Cardiologica → "CODICE 1 - EMERGENZA"
- Rosso + altro tipo → "CODICE 2 - URGENTE"
- Giallo → "CODICE 3 - PRIORITARIO"
- Verde/Bianco → "CODICE 4 - STANDARD"

Classe Clinica

- HashMap<String, Paziente> come attributo
- registraPaziente: put(cf, paziente)
- cercaPaziente: get(cf), se null → lancia eccezione
- getPazientiPerColore: itera su values() e filtra
- stampaStatistiche: conteggi + Math.round() per medie

Eccezioni custom

- Estendono Exception (checked exception)
- Hanno un costruttore con messaggio: super(msg)
- Lanciate con throw new NomeEccezione("...")
- Gestite nel Main con try { ... } catch { ... }
- Stampare e.getMessage() nel catch

Formato file: pazienti.txt

Il GestoreArchivio legge/scrive un file con questo formato. Ogni paziente è un blocco delimitato da marcatori.

```
### PAZIENTE
CF:RSSMRA85M01H501Z
NOME:Mario
COGNOME:Rossi
COLORE:Rosso
DATA_REG:2026-01-15
--- VISITA:Cardiologica
DATA:2026-01-15
MEDICO:Bianchi
DIAGNOSI:Sospetta aritmia
FREQUENZA:120
PRESSIONE_SIST:150
PRESSIONE_DIAST:95
--- FINE_VISITA
### FINE_PAZIENTE
```

Suggerimenti per il parsing

- Leggere riga per riga con BufferedReader.readLine()
- Usare line.startsWith("###") per identificare i blocchi
- Usare line.split(":") per separare chiave e valore
- Integer.parseInt(valore) per convertire numeri (Wrapper!)
- LocalDate.parse(valore) per convertire date
- Il tipo di visita si legge da "--- VISITA:Cardiologica"

Hint: Eccezioni custom

Definire l'eccezione

```
public class PazienteNonTrovatoException
       extends Exception {

  public PazienteNonTrovatoException
         (String messaggio) {
    super(messaggio);
  }
}
```

Lanciarla e catturarla

```
// In Clinica:
public Paziente cercaPaziente(String cf)
    throws PazienteNonTrovatoException {
  Paziente p = archivio.get(cf);
  if (p == null)
    throw new PazienteNonTrovat...(...);
  return p;
}
```

Nel Main (try-catch)

```
try {
    Paziente p = clinica.cercaPaziente(codiceFiscale);
    p.stampaScheda();
} catch (PazienteNonTrovatoException e) {
    System.out.println("ERRORE: " + e.getMessage());
}
```

Hint: Lettura e Scrittura File

Lettura con BufferedReader

```
BufferedReader br = new BufferedReader(
    new FileReader("pazienti.txt"));

String riga;
while ((riga = br.readLine()) != null) {
    if (riga.startsWith("### PAZIENTE")) {
        // inizio nuovo paziente
    }
    // ... parsing riga per riga
}
br.close();
```

Scrittura con PrintWriter

```
PrintWriter pw = new PrintWriter(
    new BufferedWriter(
    new FileWriter("pazienti.txt")));

for (Paziente p : archivio.values()) {
    pw.println("### PAZIENTE");
    pw.println("CF:" + p.getCF());
    // ... tutti i campi
    pw.println("### FINE_PAZIENTE");
}
pw.close();
```

Ricordatevi!

- Usate try-catch anche per IOException quando lavorate con i file!
- Integer.parseInt() converte una String in int — lancia NumberFormatException se il formato è sbagliato
- Chiudete SEMPRE reader e writer nel blocco finally o con try-with-resources

Esempio di output atteso

La stampaScheda() ora mostra TUTTE le visite del paziente:

```
╔═══════════════════════════════════════════╗
║          SCHEDA PAZIENTE - CLINICA TRIAGE       ╒
╞═══════════════════════════════════════════╡
╒ Codice Fiscale: RSSMRA85M01H501Z                ╒
╒ Paziente:       ROSSI MARIO                     ╒
╒ Triage:         ROSSO                           ╒
╒ Registrato il:  15/01/2026                      ╒
╞═══════════════════════════════════════════╡
╒ VISITA 1 — CARDIOLOGICA         15/01/2026      ╒
╒ Medico:         Dott. Bianchi                   ╒
╒ Diagnosi:       Sospetta aritmia                ╒
╒ Freq. cardiaca: 120 bpm │ Press: 150/95 mmHg    ╒
╞═══════════════════════════════════════════╡
╒ VISITA 2 — GENERICA             20/02/2026      ╒
╒ Medico:         Dott. Verdi                     ╒
╒ Diagnosi:       Controllo post-ricovero         ╒
╒ Reparto:        Medicina generale               ╒
╞═══════════════════════════════════════════╡
╒ PRIORITÀ:       CODICE 2 - URGENTE              ╒
╒ Visite totali:  2                               ╒
╚═══════════════════════════════════════════╝
```

Menu interattivo (9 comandi)

1. Registra nuovo paziente	CF + nome + cognome + colore triage. Valida CF con eccezione.
2. Cerca paziente per CF	Cerca nella HashMap. Se non trovato → PazienteNonTrovatoException.
3. Aggiungi visita a paziente	Cerca il paziente, poi crea Generica/Ortopedica/Cardiologica.
4. Stampa scheda completa	Mostra dati paziente + TUTTE le visite + priorità calcolata.
5. Elenca per codice colore	Inserisci un colore → mostra tutti i pazienti con quel colore.
6. Statistiche generali (opzionale)	Conteggio per colore, totale visite, media visite per paziente.
7. Salva archivio su file	Scrive l'archivio su pazienti.txt con PrintWriter.
8. Carica archivio da file	Legge pazienti.txt con BufferedReader e popola la HashMap.
9. Esci	Salvataggio automatico prima di uscire.